

Feature Specification

Practice Time Window File Exchange

Bamboo Health - Customer File Exchange Portal

May 2026

A serverless workflow that enables healthcare practices to configure their appointment confirmation call windows via an Excel-based file exchange. An admin generates a unique token-based URL, the practice downloads a pre-formatted Excel template, fills in their phone numbers and calling windows, and uploads the completed file. The system validates and converts the Excel data into per-practice JSON configurations that are ingested into a DynamoDB-backed API controlling when automated scheduling calls can be placed.

Requirements

1. Admin shall generate a unique 64-character token stored in DynamoDB with a 7-day TTL and email the corresponding URL to the practice via SendGrid.
2. The system shall validate the token on page load and disable all interactive elements if the token is missing, invalid, or expired.
3. The customer shall be able to download a pre-formatted Excel template via a presigned S3 GET URL (5-minute expiry).
4. The Excel template shall include: bold headers, an example row, timezone dropdown validation (5 US timezones), and input prompts on day-of-week columns specifying HH:MM-HH:MM format.
5. The customer shall be able to upload a completed .xlsx file via drag-and-drop or file browser, using a presigned S3 PUT URL (5-minute expiry) to path uploads/{token}/{filename}.
6. The conversion process shall parse the uploaded Excel workbook (sheet named 'config') and produce one JSON file per row, named {phone_without_plus}.json.
7. The converter shall validate all time values against 24-hour HH:MM format and reject the entire file if any value is invalid, reporting all errors.
8. The converter shall detect and reject duplicate phone numbers within a single upload.
9. Each JSON output shall conform to the practice-time-window schema: tz, max_days (int), attempts_per_day (int), windows[] (array of {dow, start, end}), alternate_ivr_behavior (bool), and optionally alternate_ivr_description.
10. The JSON files shall be uploaded to the practice-time-window API via PUT /config/practice-time-window/%2B{phone} using direct Lambda invocation with IAM authentication.
11. The frontend shall display inline status messages for success and descriptive error messages for failures.
12. A companion PDF shall be generated with column guide, time format rules, timezone list, and pre-submission checklist.

Non-Functional Requirements

- Security: Token-based access only. Presigned URLs expire after 5 minutes. Tokens via secrets.token_urls(48). HTTPS enforced. S3 AES-256 encryption. Referrer-Policy: no-referrer.
- Compliance: No PHI stored in the file exchange layer. Practice phone numbers and scheduling data only.
- Performance: Presigned URL generation within Lambda default timeout. Conversion handles hundreds of rows.
- Availability: Serverless architecture (Lambda + S3 + DynamoDB + CloudFront) provides inherent HA.
- Usability: Single-page interface, no login required, 5-10 minute completion time.

Technical Architecture

Stack

Component	Technology
Frontend	Static HTML/CSS/JS (vanilla), S3 + CloudFront
Backend	AWS Lambda (Python 3.12), API Gateway (HTTP API)
Storage	S3 (templates, uploads, JSON), DynamoDB (tokens)
Excel I/O	openpyxl 3.1.5
PDF Generation	fpdf2 2.8.3
AWS SDK	boto3
Email	SendGrid (planned)
IaC	Terraform
Auth	Token via DynamoDB + IAM for Lambda invocations

API Routes

Method	Path	Lambda	Purpose
GET	/validate?token=	validate_token	Check token validity
GET	/template?token=	get_template	Presigned download URL
GET	/upload-url?token=&filename=	get_upload_url	Presigned upload URL
S3	uploads/ prefix	convert_excel	Convert Excel to JSON

DynamoDB Schema (Tokens Table)

Attribute	Type	Notes
token	String (PK)	64-char via secrets.token_urlsafe(48)
customer_name	String	Practice/org name
email	String	Contact email
created_at	String	ISO timestamp
ttr	Number	Unix epoch + 7 days (DynamoDB TTL)

S3 Bucket Structure

```

template/
  practice_time_window_template.xlsx
uploads/
  {token}/
    {filename}.xlsx
json/
  {token}/
    {phone}.json
  
```

Data Flow

```

Admin generates token -> DynamoDB
Customer visits URL -> validate_token -> enable/disable UI
Download template <- get_template <- S3 presigned GET
Customer fills template offline
  
```

Upload file -> get_upload_url -> S3 presigned PUT -> S3 uploads/{token}/
S3 ObjectCreated trigger -> convert_excel Lambda
Validated JSON -> PUT /config/practice-time-window/{phone} -> DynamoDB

Build Instructions (Completed Work)

Project Structure

```
webfrontEnd/
  index.html           # Customer portal
  lambdas/get_template/handler.py  # Presigned S3 GET
  lambdas/get_upload_url/handler.py # Presigned S3 PUT
  scripts/generate_token.py        # Token generation CLI

excel2json/
  convert.py           # Excel -> JSON converter
  create_template.py   # Template generator
  generate_instructions_pdf.py # Customer PDF

api_client/
  practice_time_window_client.py  # boto3 Lambda client
```

Step 1: Excel Template Generation

File: excel2json/create_template.py | Dep: openpyxl==3.1.5

Creates practice_time_window_template.xlsx with config sheet, 13 columns, bold headers, example row, timezone dropdown, input prompts, auto-sized columns.

Run: python excel2json/create_template.py

Step 2: Customer Instructions PDF

File: excel2json/generate_instructions_pdf.py | Dep: fpdf2==2.8.3

Generates customer_instructions.pdf with column guide and formatting rules.

Run: python excel2json/generate_instructions_pdf.py

Step 3: Excel-to-JSON Conversion

File: excel2json/convert.py | Dep: openpyxl==3.1.5

Opens workbook sheet 'config', validates times via regex, checks for duplicate phones, outputs one {phone}.json per row. Fails fast on any validation error.

Run: python excel2json/convert.py <input.xlsx> [output_dir]

Step 4: API Client (Batch Upload)

File: api_client/practice_time_window_client.py | Dep: boto3

PracticeTimeWindowClient wraps Lambda invocation. put() sends single config, put_batch() iterates dict of {phone: body} pairs. IAM auth via boto3 credential chain.

Step 5: Token Generation

File: webfrontEnd/scripts/generate_token.py | Dep: boto3

Generates 64-char token, stores in DynamoDB with TTL, outputs customer URL.

Run: python webfrontEnd/scripts/generate_token.py

Step 6: Frontend Portal

File: webfrontEnd/index.html

Single-page static site. Token from URL param. Download button calls GET /template. Upload area (drag-drop + browse) calls GET /upload-url then PUTs to S3. Branding: navy #0d2137 + teal #3bbfa0, Inter font, card layout.

Step 7: Lambda - get_template

File: webfrontEnd/lambda/get_template/handler.py | Runtime: Python 3.12

Validates token in DynamoDB, generates presigned S3 GET URL (5-min), returns URL.

Step 8: Lambda - get_upload_url

File: webfrontEnd/lambda/get_upload_url/handler.py | Runtime: Python 3.12

Validates token in DynamoDB, generates presigned S3 PUT URL (5-min) for uploads/{token}/{filename}.

Next Steps (Remaining Work)

1. Terraform - DynamoDB Table

Create tokens table with token as PK (String), enable TTL on ttl attribute, on-demand billing.

2. Terraform - S3 Bucket

Single bucket with prefixes: template/, uploads/, json/. AES-256 encryption, CORS for CloudFront domain, block public access.

3. Lambda - validate_token

GET /validate?token= -> query DynamoDB -> return 200 or 403. Update index.html to call on page load.

4. Terraform - API Gateway HTTP API

Routes: /validate, /template, /upload-url. Lambda integrations. CORS headers.

5. Lambda - convert_excel

S3 trigger on uploads/ prefix. Port logic from excel2json/convert.py. Read from S3, validate, convert, write JSON to json/{token}/.

6. Terraform - S3 Event Trigger

Configure S3 notification on uploads/ prefix to invoke convert_excel Lambda.

7. Terraform - Static Site + CloudFront

S3 bucket for index.html, CloudFront distribution, Referrer-Policy: no-referrer.

8. Frontend - Set API_BASE URL

Update index.html to point to deployed API Gateway endpoint.

9. SendGrid Integration

Integrate with token generation. Send branded email with unique URL. Attach instructions PDF. (Credentials pending.)

10. End-to-End Testing

Generate token -> visit URL -> download template -> fill out -> upload -> verify JSON in S3 -> verify DynamoDB configs.

Out of Scope

- Patient-facing appointment views or confirmations (separate outreach engine)
- Real-time status tracking of conversion progress
- Admin dashboard for managing tokens or viewing submissions
- Multi-file or multi-sheet uploads
- Editing previously submitted configurations through the portal
- SMS/voice outreach execution (downstream system)
- CareNav integration and journey document management

Acceptance Criteria

1. Given a valid token URL, the page loads with download and upload controls enabled.
2. Given an invalid or expired token, the page displays an error banner and all controls are disabled.
3. Given a click on 'Download Template', the browser downloads a valid .xlsx with correct headers, example row, timezone dropdown, and input prompts.
4. Given a valid .xlsx upload with correct data, the file is stored at uploads/{token}/{filename} in S3 and a success message is displayed.
5. Given an uploaded Excel file with valid data, the converter produces one correctly-structured JSON file per row.
6. Given an uploaded Excel file with invalid time values, the converter rejects the entire file and reports all errors without writing any JSON.
7. Given an uploaded Excel file with duplicate phone numbers, the converter rejects the file and reports the duplicates.
8. Given valid JSON output files, the API client successfully PUTs each configuration to the practice-time-window endpoint.
9. Given a presigned URL request, the URL expires after 5 minutes and returns 403 on subsequent use.
10. Given a token older than 7 days, DynamoDB TTL has removed it and the system treats it as invalid.